

12. 실물 공개: 프로젝트 규칙 패턴 모음

이전 장에서는 전역 규칙의 전문을 게재했습니다.

이 장에서는 프로젝트 고유의 규칙(리포지토리 루트에 두는 CLAUDE.md)의 패턴을 소개합니다.

프로젝트 규칙은 내용이 환경에 의존하기 때문에, 전문을 실어도 참고하기 어려운 부분이 있습니다.

대신 "형(型)"으로 재사용할 수 있는 5가지 패턴을 실례와 함께 소개합니다.

패턴 1: 외부 조작의 tier 분류

조작별로 "확인 없이 OK", "확인 필요", "금지" 를 나누는 형입니다.

실례

외부 조작의 분류

확인 없이 진행해도 됨 (Tier 1)

- git commit, push (feature 브랜치)
- 로컬 npm/pip 패키지 추가·업데이트
- 테스트·lint 실행 / 개발용 서버 시작·정지
- Naver Cloud Platform / AWS 개발 환경 재배포

확인 필요 (Tier 2)

- PR 생성·머지 / DB 스키마 변경 (Prisma, Flyway)
- GitHub Actions / Jenkins 파이프라인 변경
- Kakao / Naver API 설정 변경

확인 + 신중한 실행 (Tier 3)

- 운영 서버 배포 (AWS EC2, NCP, GCP Korea Region)
- 운영 DB 마이그레이션 / Secret Manager 인증 정보 변경
- 결제·플랜 변경 / 서비스 삭제

이 패턴의 효과

처음에 "안전하게 조작해주세요"라고 썼을 때는, Claude가 git push마다 확인을 요청해 와서 하루에 10회 이상 중단이 발생했습니다.

tier 분류로 바꾼 후에는, 중단이 하루 1~2회(PR 생성이나 배포 시)로 줄었습니다.

응용 팁

- **Tier 1을 넓게 잡는다:** 로컬에서 완결되는 조작은 기본적으로 Tier 1. 망설이면 Tier 1에 넣고, 문제가 생기면 Tier 2로 올린다
- **조작명을 구체적으로 쓴다:** "외부로의 쓰기"가 아니라 "PR 생성", "배포"라고 조작명으로 쓴다
- **프로젝트마다 다르다:** 개인 프로젝트라면 push는 Tier 1, 팀 프로젝트라면 Tier 2

패턴 2: 도구의 접속 정보

Claude가 매번 조사하지 않아도 되도록, 프로젝트에서 사용하는 도구의 접속처를 써두는 형입니다.

실례

```
## 도구와 접속처

- API 서버: localhost:8080 (Spring Boot)
- 프론트엔드: localhost:3000 (Next.js)
- 데이터베이스: MySQL on localhost:3306 (Docker)
- Redis: localhost:6379 (Docker)
- CI: Jenkins (https://ci.company.com)
- 배포처: AWS EC2 (운영), NCP (스테이징)
- 이슈 트래킹: Jira BACKEND 보드
- API 문서: Swagger UI (localhost:8080/swagger-ui)
```

이 패턴의 효과

접속 정보를 쓰지 않으면, Claude는 매번 `docker ps` 나 설정 파일을 읽어 포트 번호를 조사합니다. 10초면 되는 일이지만, 1세션에서 5~10회 발생하면 누적 대기 시간이 신경 쓰입니다.

응용 팁

- **포트 번호가 특히 중요:** 기본값과 다른 포트를 사용하는 경우 Claude가 틀릴 확률이 높다
- **Docker인지 직접 설치인지 명기:** 데이터베이스가 Docker인지 로컬 설치인지에 따라 접속 방법이 달라진다
- **인증 정보는 쓰지 않는다:** 비밀번호나 토큰은 `.env` 에 두고, CLAUDE.md에는 ".env를 참조"라고만 쓴다

패턴 3: 문서의 우선순위

복수의 설정 파일이나 문서가 있을 때, 어느 것이 **정본(正本)**인지 를 명시하는 형입니다.

실례

```
## 문서의 우선순위

- 이 CLAUDE.md가 최상위 설정. 다른 문서와 모순될 경우 이쪽에 따른다
- API 사양: OpenAPI 정의 (api/openapi.yaml)가 정본. README 기술이 오래된 경우가 있다
- 배포 절차: DEPLOYMENT.md가 정본. Wiki 절차는 업데이트가 늦어져 있을 가능성 있음
- 환경 변수: .env.example이 정의 목록. 실제 값은 .env (Git 관리 외)
```

이 패턴의 효과

프로젝트가 성장하면, 같은 정보가 여러 곳에 쓰여집니다. README, Wiki, CLAUDE.md, 코드 내 주석..... Claude는 어느 것을 믿어야 할지 망설입니다.

"정보는 이것"이라고 명시해두면, Claude는 모순되는 정보를 만났을 때 정보를 우선합니다.

패턴 4: 병렬 작업의 규칙

서브에이전트나 복수 세션으로 작업할 때의 제약을 쓰는 형입니다.

실례

병렬 작업의 규칙

- 다른 세션이 편집 중일 가능성이 있는 파일을 덮어쓰지 않는다
 - `git status`로 `uncommitted changes`를 확인하고 나서 편집한다
- `git add -A`나 `git reset --hard`를 사용하지 않는다
 - 다른 세션의 `staged changes`를 포함할 수 있다
- `shared/` 디렉토리의 파일은 병렬 편집하지 않는다
 - 공통 타입 정의나 유틸리티는 한 사람씩 건드린다
- MCP 서버의 재시작은 다른 세션의 접속을 끊는다
 - 재시작이 필요할 때는 영향을 보고한다

이 패턴의 효과

혼자서 작업할 때는 신경 쓰지 않아도 되는 규칙이지만, 서브에이전트를 사용하기 시작하면 필수가 됩니다. 2개의 서브에이전트가 같은 파일을 동시에 편집해서 한쪽의 변경이 사라지는 사고를 방지합니다.

응용 팁

- 병렬화하지 않는 파일을 명시: "`shared/`는 한 사람씩" 처럼 병렬 금지 디렉토리를 쓴다
- git의 위험 명령어를 열거: `git add -A`, `git reset --hard`, `git clean -f` 는 병렬 작업에서 특히 위험하다

패턴 5: 정지 시 보고 포맷

Claude가 작업을 중단할 때(사용자 확인이 필요할 때)의 보고 형식을 지정하는 형입니다.

실례

정지 시 보고 포맷

작업이 정지 경계에 도달했을 때, 다음 형식으로 보고한다:

필요 액션: [무엇을 할지. 예: PR을 생성해도 되는가]

대상: [무엇에 대해. 예: `my-app/main`]

완료한 작업: [여기까지 한 내용]

다음 단계: [확인 후 할 것]

- "필요 액션:"은 정말로 사용자 조작이 남아있을 때만 사용한다
- 사용자 조작이 필요 없으면 "필요 액션: 없음"이라고 쓰고 작업을 계속한다

이 패턴의 효과

포맷을 정하지 않으면, Claude의 보고는 장황해지기 쉽습니다.
포맷을 정하면, 첫 번째 줄에서 "무엇을 하면 되는가" 를 알 수 있습니다.

5가지 패턴의 조합

이 패턴들은 독립적으로도 사용할 수 있지만, 조합하면 더 효과적입니다.

미니멀한 프로젝트 CLAUDE.md 구성:

```
## 외부 조作的 분류      ← 패턴 1
## 도구와 접속처          ← 패턴 2
## 문서의 우선순위        ← 패턴 3
```

풀 스펙 구성 (서브에이전트 사용, 팀 공유):

```
## 프로젝트 개요
## 외부 조作的 분류      ← 패턴 1
## 도구와 접속처          ← 패턴 2
## 문서의 우선순위        ← 패턴 3
## 병렬 작업의 규칙      ← 패턴 4
## 정지 시 보고          ← 패턴 5
```

모든 패턴을 처음부터 사용할 필요는 없습니다.
패턴 1(tier 분류)만으로도 충분한 효과가 있습니다. 필요해지면 추가하세요.